

SOLITARIO INGLESE

Scopo del progetto

Date le regole del solitario inglese:

- Scacchiera a croce con 33 caselle
- Su ogni casella è presente una biglia tranne nella casella centrale che dev'essere vuota
- Mosse ammissibili:

Si può spostare una pedina di esattamente due caselle sulla scacchiera

- in verticale (in alto o in basso)
- in orizzontale (a sinistra o a destra)

se e solo se la casella d'arrivo è libera e la casella saltata è occupata.

Infine si elimina la pedina nella casella spostata

- Condizione di vittoria:

- Sulla scacchiera rimane una sola pedina
- La pedina è nella posizione centrale

Creare un programma che, considerando un giocatore che fa mosse ammissibili casuali, calcoli:

- La probabilità in cui vinca
- La probabilità in cui il gioco termini in una configurazione in cui rimangono esattamente k pedine, con $k \in \{1, \dots, 32\}$
- La probabilità abbia durata di esattamente N mosse con $N \in \{1, \dots, 31\}$
- La durata media del gioco (in n. mosse)
Numero medio di pedine nella configurazione terminale

Giustificazione matematica per le formule utilizzate

Def: $N :=$ durata del gioco in mosse, $N \in \{1, \dots, 31\}$

$k := 32 - N =$ numero di pedine nella configurazione terminale, $k \in \{1, \dots, 32\}$

$V = 1(K=1 \text{ and } S_r(c^*)=1) =$ indicatore di vittoria

$n :=$ numero di partite

$$a) P(\text{vittoria}) = \frac{\#\{i: V_i=1\}}{n} \quad (\text{partite vinte su partite totali})$$

$$b) P(K=k) = \frac{\#\{i: K_i=k\}}{n}, \quad k \in \{1, \dots, 32\}$$

$$c) P(N=m) = \frac{\#\{i: N_i=m\}}{n}, \quad m \in \{1, \dots, 31\}$$

$$d) E(N) = \frac{1}{n} \sum_{i=1}^n N_i$$

$$e) E(K) = \frac{1}{n} \sum_{i=1}^n K_i = 32 - E(N)$$

Programma:

Scacchiera.h:

scacchiera è griglia a croce 7×7 in cui solo 33 caselle sono valide: (Rappresentata dalla classe pos)

$$C = \{(r, c) \in \{0, \dots, 6\}^2 \mid (r \in \{2, 3, 4\}) \vee (r \notin \{2, 3, 4\} \wedge c \in \{2, 3, 4\})\}$$

casella centrale è $C^* = (3, 3)$

Una configurazione è definita da una funzione $s: C \rightarrow \{0, 1\}$, dove $s(c) = 1$ indica casella c è occupata da una pedina e $s(c) = 0$ indica c libera (rappresentata dalla classe board)

Lo stato iniziale:

$$s_0(c) = \begin{cases} 0 & p = c^* \\ 1 & p \neq c^* \end{cases}$$

-Classi:

- pos:
 int r, c
- board
 bool cell[7][7]

Metodi:

pos:

```
bool is_valid(int r, int c) // controlla se la posizione (r, c) è scacchiera
if r < 0 or r > 6 or c < 0 or c > 6
    return false;
if r < 2 or r > 4
    return c > 1 and c < 5
return true;
```

```
void init_valid() // inizializza array valid_pos[33] con le posizioni valide
n_valid = 0;
for i = 0 to 7 do
    for j = 0 to 7 do
        if is_valid(i, j)
            valid_pos[n_valid + 1] ← pos(i, j);
```

board:

```
void init() // inizializza la tabella con configurazione iniziale
cells[1...7][1...7] ← 0
for i = 0 to n_valid // con n_valid = 33
    p ← valid_pos[i]
    set(p, !(p = centro))
```

```
void set(pos p, bool val) // pone la casella in posizione p sulla griglia uguale al valore val
cells[p.r][p.c] = val;
```

```
void get(pos p) // torna il valore della casella in posizione p sulla griglia
return cells[p.r][p.c]
```

```
int count() // conta il numero di biglie rimaste sulla scacchiera
```

```
n=0
```

```
for i=0 to n-valid do  
  if get(valid_pos[i])  
    n+1
```

```
return n
```

```
bool victory () // torna true in caso di vittoria  
return count()==1 and get(centro);
```

mosse.h:

Una mossa è descritta da:

pos from=(fr, fc) posizione della pedina da spostare, dev'essere true

pos over=(or, oc) posizione della pedina da eliminare, dev'essere true

pos to=(tr, tc) posizione casella d'arrivo, dev'essere false

Le 4 direzioni di spostamento possibili sono: su (-2,0), giù (2,0), sx (0,-2), dx (0,+2)

Classi:

- move:

pos from, over, to

Metodi:

move:

void apply(board b) // applica una mossa

b.set(from, false)

b.set(over, false)

b.set(to, true)

Funzioni:

```
int get_moves(board b, move m[132]) // torna il numero di mosse ammissibili per una  
int n=0 // configurazione b; e salva in un array m  
for i=0 to n-valid do // le mosse ammissibili
```

```
pos from = valid_pos[i]
```

```
if (!b.get(from)) continue // controlla se nella posizione from c'è biglia
```

```
for pos d : dirs // dirs[4] = {(-2,0), (2,0), (0,-2), (0,2)}
```

```
pos over ← from + pos(d.r/2, d.c/2) // calcolo over e to
```

```
pos to ← from + d
```

```
if !over.isvalid or !b.get(over) // controllo se pos. over e to sono  
continue // all'interno della scacchiera; e se
```

```
if !to.isvalid or b.get(to) // over è true e to è false  
continue
```

```
m[n+1] = {from, over, to}
```

partita.h:

Salva i risulta di una partita

-Classi:

-result:

```
int n_moves;  
int n_pegs;  
bool victory;
```

Metodi:

```
mt19937 rng() // generatore di seed pseudo randomico
```

```
result play_random_game(mt19937 rng) // prende il seed randomico e genera  
// una partita
```

```
while true // ciclo finché non arriva a n=0  
    int n = get(moves(board, moves) // calcola n mosse ammissibili  
    if n == 0  
        break
```

```
uniform_int_distribution<int> dist(0, n-1)  
moves[dist(rng)].apply(board) // genera numero pseudo randomico  
// tra 0 e n-1 (numero di mosse  
// ammissibili) a partire dal seed e  
// lo applica  
n_moves + 1
```

```
return result(n_moves, board.count(), board.victory());
```

risultati.h:

Salva i risultati di n-sim (numero di partite) e calcola:

a) b) c) d) e)

-classi:

-MCResult:

```
int n_sim // n. partite giocate  
long n_victory // n. vittorie  
long count_k[33] // n. partite terminate con k pedine  
long count_N[32] // n. partite terminate con N mosse  
double total_moves // n. totale di mosse fatte (per calc. media mosse per  
// partita)  
double total_pegs // n. totale di biglie rimaste (per calc. media biglie per  
// partita)
```

-Metodi:

```
void reset() // resetta i risultati salvati (per avviare nuova simulazione)
```

```
void add(result r) // somma i vari risultati delle simulazioni e li ritorna
```

```
n_sim + 1  
if r.victory  
    n_victory + 1
```

```
if r.n_pegs ≥ 1 and r.n_pegs ≤ 32  
    count_k[r.n_pegs] += 1
```

```
if r.n_moves ≥ 1 and r.n_moves ≤ 32  
    count_N[r.n_moves] += 1
```

```
total_moves += r.n_moves  
total_pegs += r.n_pegs
```

```
void run(int n, mt19937 rng) // gioca n partite con seed pseudo random rng  
    for i=0 to n do           // e ne salva i risultati  
        add(play_random_game(rng))
```

```
void print() // stampa a schermo i risultati
```

```
void export_csv() // salva i risultati in un file csv per costruire i grafici
```